فصل ۸: تفکر مؤلفه‌محور

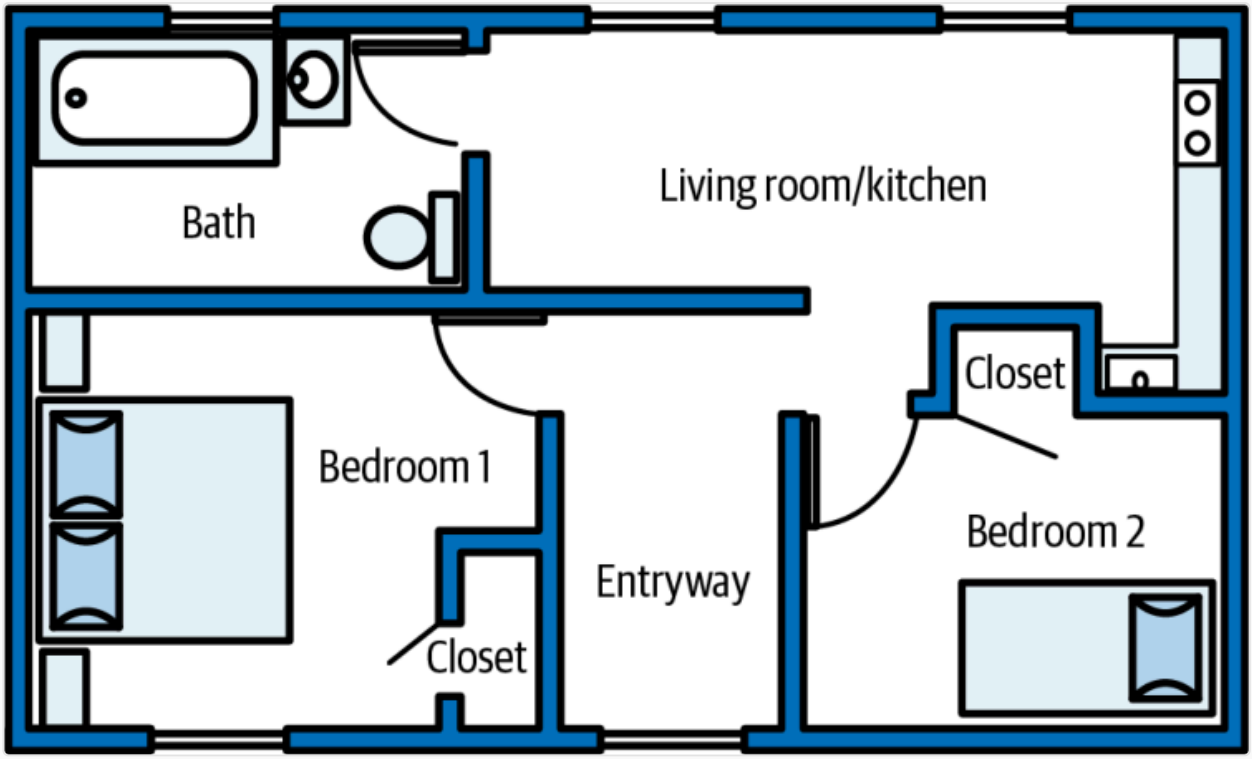
در فصل ۳ مفهومِ ماژول را به‌عنوانِ مجموعه‌ای از کدهای مرتبط معرفی کردیم. در این فصل بسیار عمیق‌تر به این مفهوم می‌پردازیم و بر جنبهٔ معماریِ ماژولاریتی در قالبِ مؤلفه‌های منطقی، یعنی بلوک‌های سازندهٔ یک سیستم، تمرکز می‌کنیم.

شناسایی و مدیریتِ مؤلفه‌های منطقی بخشی از تفکر معماری است (فصل ۲ را ببینید)؛ آن‌قدر که این فعالیت را تفکر مؤلفه‌محور `(component-based thinking)` می‌نامیم. تفکر مؤلفه‌محور یعنی ساختارِ سیستم را مجموعه‌ای از مؤلفه‌های منطقی ببینیم که همگی برای انجامِ کارکردهای کسب‌وکاریِ مشخصی با هم تعامل دارند. معمار سیستم را در همین سطح (و نه در سطحِ کلاس) «می‌بیند».

در این فصل، مؤلفه‌های منطقی را در معماری نرم‌افزار تعریف می‌کنیم و نشان می‌دهیم چگونه می‌توان آن‌ها را شناسایی کرد و با تحلیلِ چیزی به نامِ انسجام، به سطحِ مناسبی از دانه‌بندی رسید (معنای انسجام را کمی بعدتر در همین فصل تعریف می‌کنیم). همچنین دربارهٔ کوپلینگ میانِ مؤلفه‌ها بحث می‌کنیم و اینکه چگونه و چرا باید سیستم‌هایی با کوپلینگ سست ساخت.

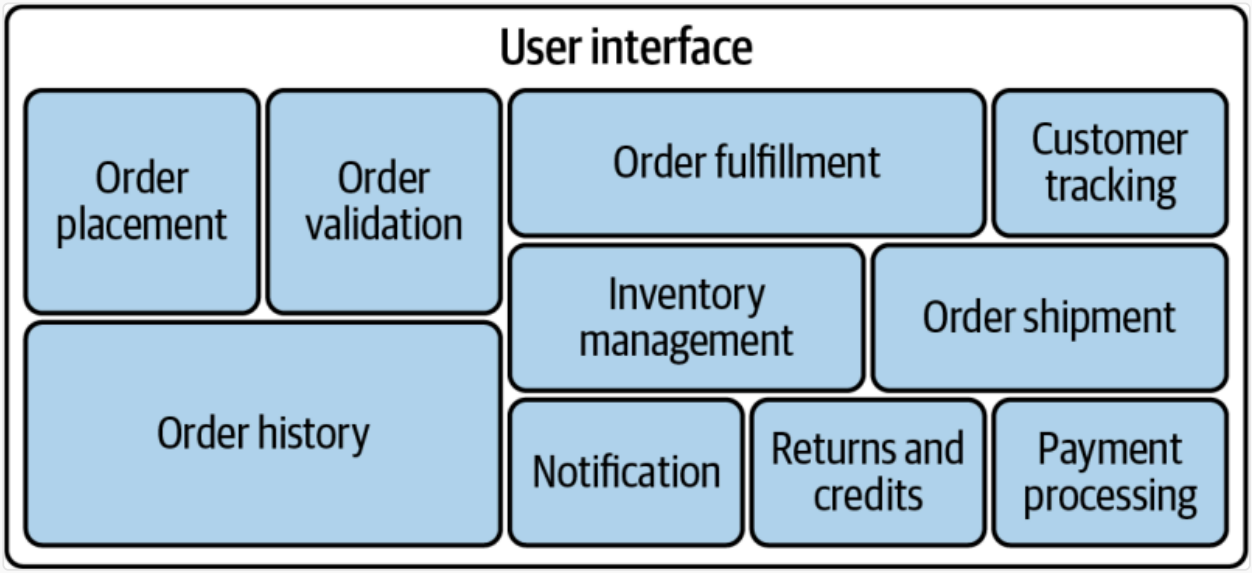
تعریفِ مؤلفه‌های منطقی

نقشهٔ طبقهٔ `(floor plan)` یک خانهٔ معمولیِ غربی را، چنان‌که در شکل ۸-۱ نشان داده شده، در نظر بگیرید. می‌بینید که این نقشه از اتاق‌های گوناگونی (مانندِ آشپزخانه، اتاق‌خواب‌ها، حمام‌ها، اتاق نشیمن، اتاق کار و جز آن) تشکیل شده که هر یک کاربردِ متفاوتی دارد. این اتاق‌ها بلوک‌های سازنده، یعنی مؤلفه‌های، خانه را نشان می‌دهند.



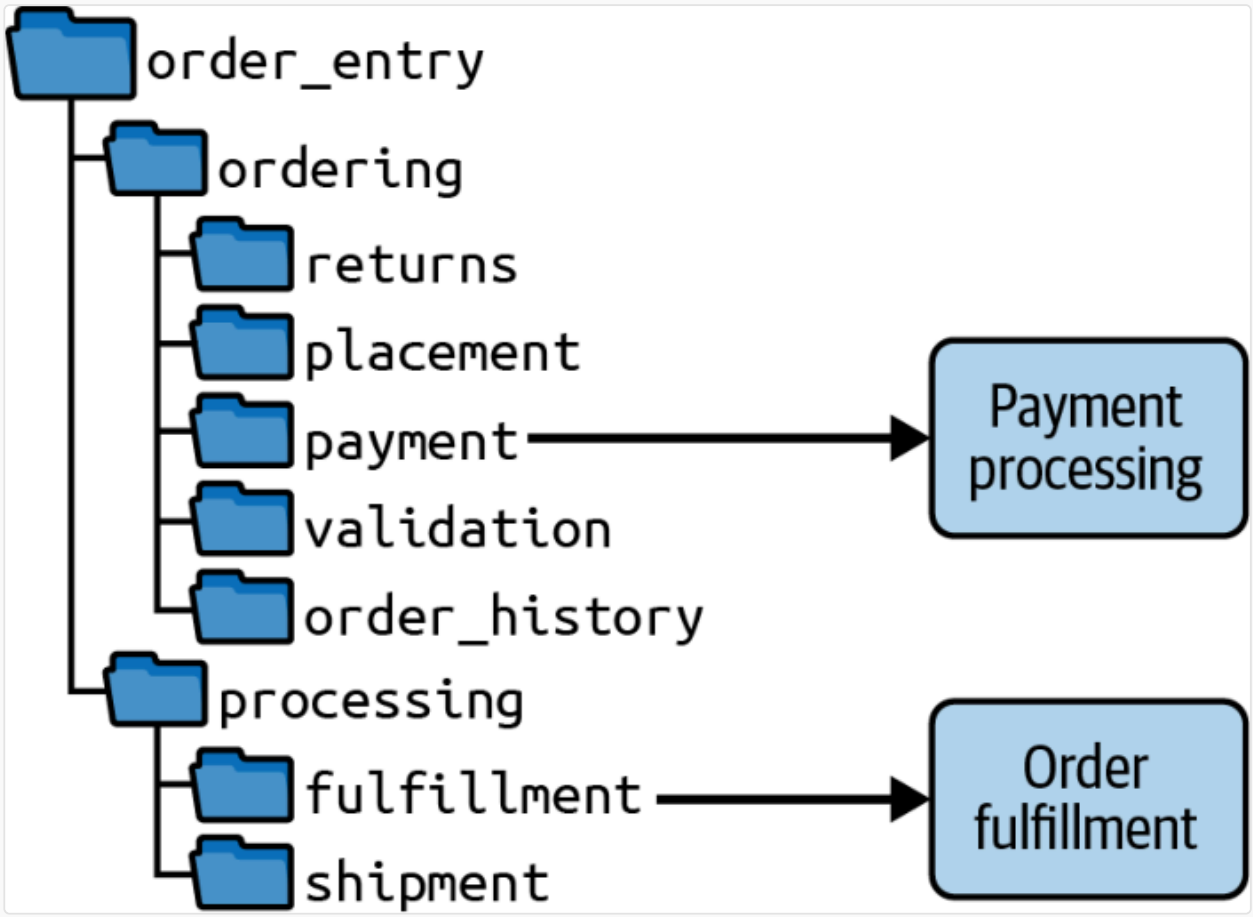
شکل ۸-۱. اتاق‌های مختلف، مؤلفه‌های خانه را نشان می‌دهند

به همین ترتیب، کارکردهای اصلی‌ای که یک سیستم انجام می‌دهد، مؤلفه‌های آن سیستم را نشان می‌دهند؛ چنان‌که در شکل ۸-۲ می‌بینید. هر مؤلفه، مانندِ اتاق‌های یک خانه، کارکردِ مشخصی انجام می‌دهد؛ مانندِ مدیریتِ موجودیِ انبار، ارسالِ سفارش‌ها یا پردازشِ پرداخت‌ها. این مؤلفه‌ها روی هم سیستم را می‌سازند. هر مؤلفه کدِ منبعی را در بر دارد که آن کارکردِ کسب‌وکاریِ خاص را پیاده می‌کند.



شکل ۸-۲. کارکردهای اصلیِ مختلف، مؤلفه‌های سیستم را نشان می‌دهند

مؤلفه‌های منطقی در معماری نرم‌افزار معمولاً از راهِ یک فضای نام یا ساختارِ دایرکتوری‌ای نمود می‌یابند که کدِ منبعِ پیاده‌کنندهٔ آن کارکردِ خاص را در بر دارد. به‌طورِ معمول، گره‌های برگِ `(leaf nodes)` ساختارِ دایرکتوری یا فضای نامی که کدِ منبع را در بر دارد، مؤلفه‌های منطقیِ معماری را نشان می‌دهند و دایرکتوری‌ها یا گره‌های فضای نامِ سطحِ بالاتر، حوزه‌ها و زیرحوزه‌های `(subdomains)` سیستم را. در ساختارِ دایرکتوریِ نشان‌داده‌شده در شکل ۸-۳، مسیرِ دایرکتوریِ `order_entry/ordering/payment` مؤلفهٔ `Payment Processing` را نشان می‌دهد و مسیرِ `order_entry/processing/fulfillment` مؤلفهٔ `Order Fulfillment` را. کدِ منبعِ زیرِ این دایرکتوری‌ها این مؤلفه‌های منطقی را پیاده می‌کند.

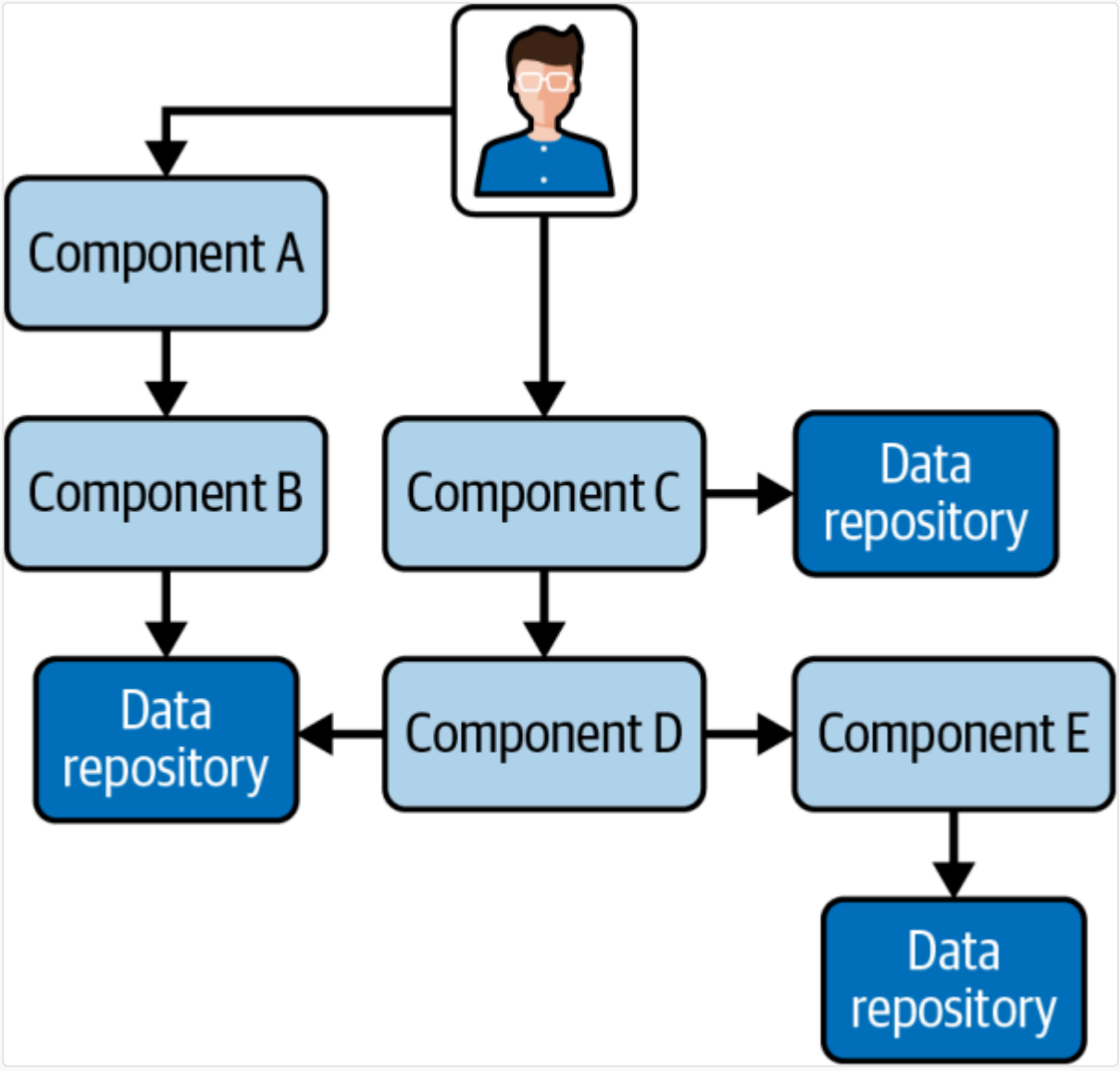


شکل ۸-۳. گره‌های برگِ دایرکتوری، مؤلفه‌های سیستم را نشان می‌دهند

معمار می‌تواند ساختارهای دایرکتوری یا فضاهای نامِ یک سیستمِ نرم‌افزاری را تحلیل کند تا ساختارِ درونیِ آن، یا به بیانِ دیگر معماری منطقیِ `(logical architecture)` آن، را بفهمد. تفاوت‌های معماری منطقی و معماری فیزیکی را در بخشِ بعد شرح می‌دهیم.

معماری منطقی در برابرِ معماری فیزیکی

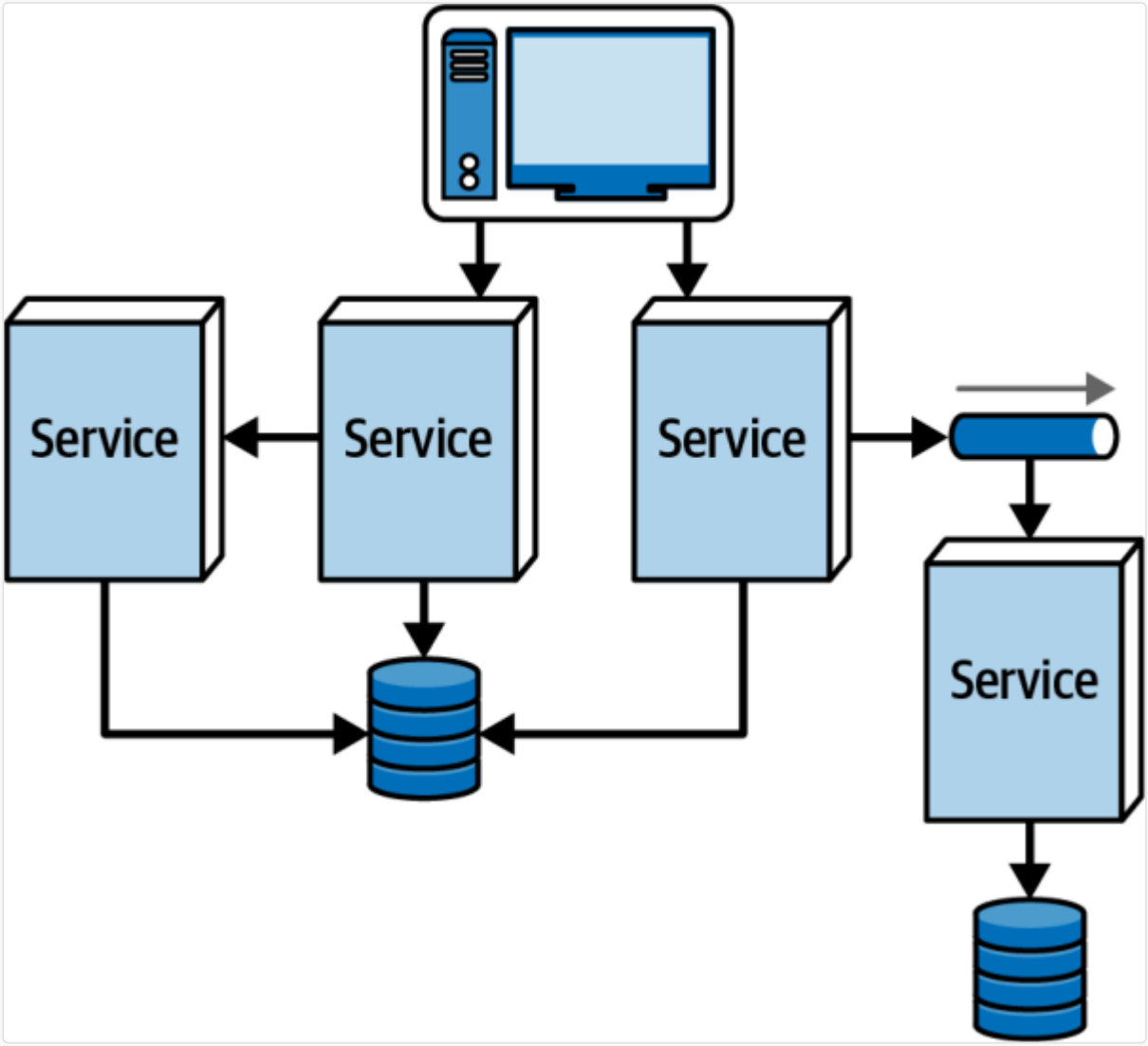
معماری منطقی از مؤلفه‌های منطقیِ (بلوک‌های سازندهٔ) سیستم و شیوهٔ تعاملِ آن‌ها با یکدیگر تشکیل می‌شود. این معماری معمولاً تعاملِ مؤلفه‌ها با کنشگرانِ `(actors)` گوناگون (کاربرانی که با سیستم تعامل دارند) را نیز نشان می‌دهد و ممکن است برای روشن‌کردنِ اینکه داده‌ها کجا به کار می‌روند یا میانِ مؤلفه‌ها جابه‌جا می‌شوند، یک مخزن `(repository)`، و نه پایگاه‌داده، را هم نمایش دهد. شکل ۸-۴ نمونه‌ای از معماری منطقی را نشان می‌دهد.



شکل ۸-۴. نمودارِ یک معماری منطقی

نمودارِ معماری منطقی معمولاً رابط کاربری، پایگاه‌های داده، سرویس‌ها یا دیگر مصنوعاتِ فیزیکی `(physical artifacts)` را نشان نمی‌دهد. در عوض، مؤلفه‌های منطقی و شیوهٔ تعاملِ آن‌ها را نشان می‌دهد؛ شیوه‌ای که باید با ساختارهای دایرکتوری و فضاهای نامی که کد را سازمان می‌دهند مطابقت داشته باشد.

در مقابل، معماری فیزیکی `(physical architecture)` مصنوعاتِ فیزیکی‌ای مانندِ سرویس‌ها، رابط‌های کاربری، پایگاه‌های داده و مانندِ آن را در بر می‌گیرد. شکل ۸-۵ نمونه‌ای از نمودارِ معماری فیزیکی است.



شکل ۸-۵. نمودارِ یک معماری فیزیکی

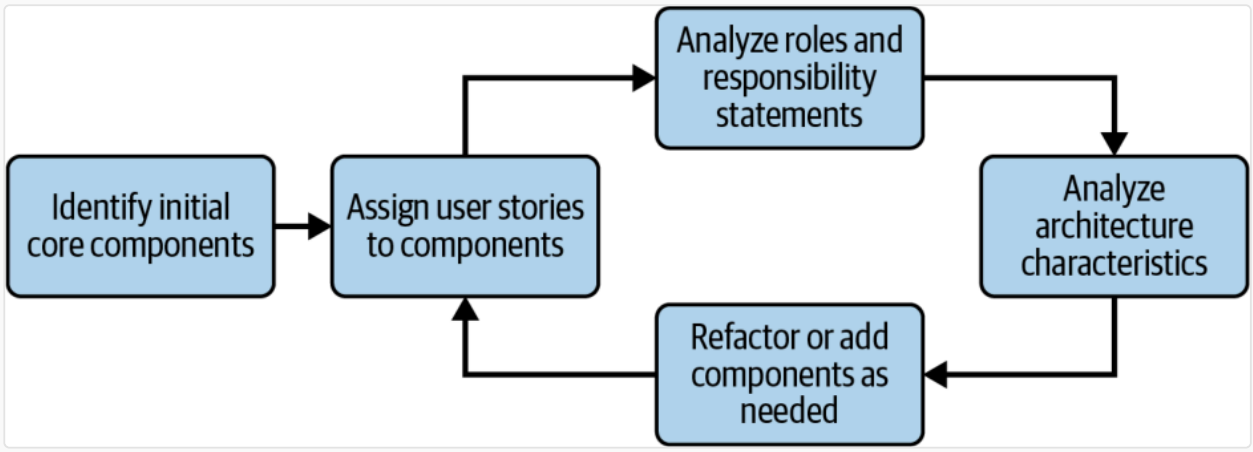
معماری فیزیکیِ سیستم باید به‌دقت بازتاب‌دهندهٔ یک (یا چند) سبک از سبک‌های معماریِ متعددی باشد که در بخشِ دوم این کتاب شرح داده شده‌اند: میکروسرویس‌ها، معماری لایه‌ای، معماری رویدادمحور و جز آن.

بسیاری از معماران از ساختنِ معماری منطقی چشم می‌پوشند و مستقیماً با معماری فیزیکی آغاز می‌کنند. اما ما این شیوه را توصیه نمی‌کنیم، زیرا معماری فیزیکی همیشه نشان نمی‌دهد کارکردهای سیستم کجا قرار دارند و چگونه در کنارِ هم جای می‌گیرند. برای نمونه، در یک معماری فیزیکی، کارکردِ پردازشِ پرداخت ممکن است در چندین سرویس پخش شده باشد و دیدنِ نحوهٔ تعاملِ آن با دیگر بخش‌های سیستم را دشوار کند. افزون بر این، معماری فیزیکی هیچ راهنمایی‌ای به تیم‌های توسعه نمی‌دهد که چگونه یک سیستمِ تک‌پارچه یا توزیع‌شده بسازند یا کدِ آن را سازمان دهند؛ نتیجه، عمدتاً معماری‌هایی بی‌ساختار است که نگهداری، آزمون و استقرارشان دشوار است.

به‌طورِ کلی، معماری منطقیِ سیستم مستقل از معماری فیزیکیِ آن است. به بیانِ دیگر، هنگامِ ساختنِ معماری منطقی، تمرکز بیشتر بر این است که سیستم چه می‌کند، مرزِ آن کارکردها چگونه تعیین می‌شود و بخش‌های کارکردیِ سیستم چگونه با هم تعامل دارند؛ نه بر ساختارِ فیزیکیِ آن. برای نمونه، معماری که معماری منطقی‌ای مانندِ شکل ۸-۴ می‌سازد، ممکن است هنوز تصمیم نگرفته باشد که آیا همهٔ آن مؤلفه‌ها در یک معماری تک‌پارچه، یعنی یک واحد استقرار `(deployment unit)`، قرار می‌گیرند یا به‌صورتِ سرویس‌ها، یعنی واحدهای استقرارِ جداگانه، مستقر می‌شوند؛ و لزوماً هم تعیین نکرده باشد که چه نوع سبکِ معماری‌ای مناسب‌تر است.

ساختنِ معماری منطقی

ساختنِ معماری منطقی مستلزمِ شناسایی و بازساختاردهیِ پیوستهٔ مؤلفه‌های منطقی است. شناسایی مؤلفه‌ها `(component identification)` به‌صورتِ فرایندی تکرارشونده بهترین نتیجه را می‌دهد. این فرایند شاملِ تولیدِ مؤلفه‌های نامزد `(candidate components)` و سپس پالایشِ آن‌ها از راهِ یک حلقهٔ بازخورد `(feedback loop)` است؛ چنان‌که در شکل ۸-۶ نشان داده شده است.



شکل ۸-۶. چرخهٔ شناسایی و بازآرایی مؤلفه‌ها

نخستین فعالیتِ معمار در توسعهٔ معماری منطقی، شناساییِ مؤلفه‌های اصلیِ `(core components)` اولیه و سپس تخصیصِ داستان‌های کاربری یا الزامات به آن‌هاست. پس از آن، نقش‌ها و مسئولیت‌های هر مؤلفه را تحلیل می‌کند تا مطمئن شود داستان‌های کاربری یا الزاماتی که به آن تخصیص یافته‌اند معنادارند. سپس معمار به ویژگی‌های معماری‌ای که سیستم باید پشتیبانی کند می‌نگرد و تعیین می‌کند که آیا بر پایهٔ این ویژگی‌ها به بازآرایی `(refactoring)` نیاز است تا مؤلفه احتمالاً شکسته یا با مؤلفه‌ای دیگر ترکیب شود. سرانجام، معمار در صورتِ لزوم مؤلفه‌ها را بر پایهٔ این تحلیل پالایش می‌کند. این فرایند حلقهٔ بازخوردی است که اساساً هرگز متوقف نمی‌شود.

گردش‌کارِ شکل ۸-۶ را می‌توان برای سیستم‌های نوساخت `(greenfield)` یا هر زمان که قابلیتی به سیستم افزوده یا در آن تغییر داده می‌شود به کار برد. برای نمونه، فرض کنید باید یک سیستمِ ثبتِ سفارش را طوری تغییر دهید که کاربران بتوانند اقلام را به‌جای ارسال به خانه، از فروشگاه تحویل بگیرند. این کار مستلزمِ افزودنِ کدِ زمان‌بندی و نیز

تغییرِ فرایندِ سفارش‌دهیِ موجود است. این تغییر ممکن است به مؤلفه‌های جدید، تغییر در مؤلفه‌های موجود، یا هر دو نیاز داشته باشد. با تغییرِ مؤلفه‌ها، نقش‌ها و مسئولیت‌هایشان نیز ممکن است تغییر کند و شما را به بازساختاردهیِ کد یا ساختنِ مؤلفه‌های جدید برای آن وادارد.

هر یک از این گام‌ها را در بخش‌های بعدی به‌تفصیل شرح می‌دهیم.

شناساییِ مؤلفه‌های اصلی

چالشِ آغازِ یک معماری منطقیِ جدید (یا اعمالِ تغییراتِ عمده در معماریِ موجود) این است که تعیین کنیم مؤلفه‌های اصلیِ اولیه چه باید باشند. یکی از اشتباه‌هایی که بسیاری از معماران نرم‌افزار مرتکب می‌شوند، صرفِ تلاشِ بیش از اندازه برای بی‌نقص‌کردنِ مؤلفه‌های منطقیِ اولیه در همان نخستین بار است. رویکردِ بهتر این است که بر پایهٔ کارکردهای اصلیِ سیستم «بهترین حدس» را دربارهٔ شکلِ احتمالیِ مؤلفه‌های اصلیِ اولیه بزنیم و آن‌ها را با گام‌های گردش‌کارِ ترسیم‌شده در شکل ۸-۶ پالایش کنیم. به بیانِ دیگر، بهتر است همان‌طور که دربارهٔ سیستم بیشتر می‌آموزید، مؤلفه‌های منطقی را به‌صورتِ تکرارشونده بهبود دهید؛ نه آنکه بکوشید همه‌چیز را در نخستین بار، یعنی زمانی که کمترین شناخت را از سیستم و الزاماتِ خاصِ آن دارید، بی‌نقص کنید.

در بیشترِ موارد، معمار برای آغازِ ساختنِ مؤلفه‌های منطقی نیازی ندارد همهٔ الزامات و مشخصاتِ سیستم (یا گاهی هیچ‌یک از آن‌ها) را بداند. به‌طورِ معمول، مؤلفه‌های اصلیِ اولیه بر پایهٔ کنش‌های عمده‌ای که کاربران ممکن است انجام دهند یا گردش‌کارهای پردازشیِ عمدهٔ سیستم شکل می‌گیرند.

دوست داریم مؤلفه‌های اصلیِ اولیه را سطل‌هایی خالی تصور کنیم. سطل تا زمانی که معمار شروع به «پرکردنِ» آن، یعنی تخصیصِ داستان‌های کاربری یا الزامات به آن مؤلفه، نکند کاری انجام نمی‌دهد. چنان‌که در شکل ۸-۷ نشان داده شده، نامِ مؤلفه باید نقش و مسئولیتِ پیشنهادیِ آن را بازتاب دهد. اما تا زمانی که معمار داستان‌های کاربری را به آن تخصیص ندهد (گامِ بعدیِ گردش‌کار)، مؤلفه در اصل یک جانگهدار `(placeholder)` است؛ بهترین حدس دربارهٔ یک بلوکِ سازندهٔ کارکردی.



شکل ۸-۷. در آغاز، مؤلفه‌های اولیه مانندِ سطل‌هایی خالی‌اند

در ادامه سه رویکردِ رایج برای ساختنِ مؤلفه‌های اصلیِ اولیه آمده است. دو رویکردِ نخست، یعنی رویکردِ گردش‌کار (Workflow) و رویکردِ کنشگر/کنش (Actor/Action)، را سودمند می‌دانیم؛ اما سومی، یعنی تلهٔ موجودیت (Entity Trap)، ضدالگویی است که از آن برحذر می‌داریم.

رویکردِ گردش‌کار

یکی از رویکردهای رایجی که معماران برای شناساییِ مؤلفه‌های اصلیِ اولیهٔ یک معماری منطقی به کار می‌برند، رویکرد گردش‌کار است. همان‌طور که از نامش پیداست، این رویکرد از گردش‌کارهای اصلیِ مسیرِ موفق (happy path)، یعنی گردش‌کارهای بدونِ خطایی که کاربر ممکن است در سیستم طی کند (یا گردش‌کارِ اصلیِ پردازشِ درخواست‌ها)، بهره می‌گیرد. اگر معمار تصوری کلی از این جریان داشته باشد، می‌تواند مؤلفه‌ها را بر پایهٔ گام‌های آن بسازد.

برای نمونه، فرض کنید در حالِ ساختنِ یک سیستمِ ثبتِ سفارشِ جدید برای شرکتتان هستید. هنوز الزامات و مشخصاتِ دقیق را نمی‌دانید، اما دست‌کم گردش‌کارِ کلیِ پردازشِ یک سفارشِ جدید را می‌شناسید. بنابراین می‌توانید به هر گامِ گردش‌کار یک مؤلفه تخصیص دهید:

1. کاربر کاتالوگِ اقلام را مرور می‌کند ← `Item Browser`
2. کاربر سفارش ثبت می‌کند ← `Order Placement`
3. کاربر هزینهٔ سفارش را می‌پردازد ← `Order Payment`
4. ایمیلی حاویِ جزئیاتِ سفارش برای کاربر ارسال می‌شود ← `Customer Notification`
5. سفارش آماده می‌شود ← `Order Fulfillment`
6. سفارش ارسال می‌شود ← `Order Shipment`
7. به مشتری ایمیل زده می‌شود که سفارش ارسال شده است ← `Customer Notification`
8. محموله ردیابی می‌شود ← `Order Tracking`

در رویکرد گردش‌کار، هر گامِ گردش‌کارِ اصلی لزوماً مؤلفهٔ جدیدی پدید نمی‌آورد. در گردش‌کارِ بالا، گام‌های ۴ و ۷ هر دو از مؤلفهٔ `Customer Notification` استفاده می‌کنند. معمار هر تعداد گردش‌کارِ اصلی یا سفرِ کاربری (user journey) را که بتواند برای سیستم مدل می‌کند و مؤلفه‌های متناظر را از گام‌های آن‌ها شناسایی می‌کند.

این نیز، باز هم، تنها «بهترین حدس» دربارهٔ شکلِ احتمالیِ معماری منطقی است. این مؤلفه‌ها سطل‌هایی خالی‌اند و هنوز مسئولیتی ندارند؛ بنابراین به احتمالِ زیاد در طولِ تکاملشان تغییر خواهند کرد (چنان‌که در شکل ۸-۶ نشان داده شد). این کاملاً طبیعی و بخشی از ماهیتِ تکرارشوندهٔ معماری نرم‌افزار است. نگرانِ مدل‌کردنِ همهٔ گردش‌کارهای سیستمتان نباشید. در عوض، بر گردش‌کارهای اصلی تمرکز کنید. بقیه همان‌طور که دربارهٔ سیستم بیشتر می‌آموزید و گردآوریِ داستان‌های کاربری و الزامات را آغاز می‌کنید، شکل می‌گیرند.

رویکردِ کنشگر/کنش

راهِ دیگری که معماران برای شناساییِ مؤلفه‌های اصلیِ اولیه به کار می‌برند، رویکرد کنشگر/کنش است. این رویکرد به‌ویژه زمانی سودمند است که سیستم چندین کنشگر داشته باشد. در این رویکرد، معمار کنش‌های اصلی‌ای را که کاربر می‌تواند در سیستم انجام دهد (مانندِ ثبتِ سفارش) شناسایی می‌کند. خودِ سیستم نیز همیشه یک کنشگر است که کارکردهای خودکاری مانندِ صدورِ صورت‌حساب و تأمینِ دوبارهٔ موجودیِ انبار را انجام می‌دهد.

در مثالِ سیستمِ ثبتِ سفارشِ ما، معمار ممکن است سه کنشگر را شناسایی کند: مشتری، بسته‌بندِ سفارش (`packer`) (کسی که جعبه را بسته‌بندی می‌کند و برای ارسال می‌فرستد) و سیستم. سپس معمار کنش‌های اصلیِ هر یک از این کنشگران را شناسایی می‌کند و به آن کنش‌ها مؤلفه تخصیص می‌دهد:

کنشگرِ مشتری:

- جست‌وجوی اقلام ← `Item Search`
- مشاهدهٔ جزئیاتِ یک قلم کالا ← `Item Details`
- ثبتِ سفارش ← `Order Placement`
- لغوِ سفارش ← `Order Cancel`
- ثبت‌نام به‌عنوانِ مشتریِ جدید ← `Customer Registration`
- به‌روزرسانیِ اطلاعاتِ مشتری ← `Customer Profile`

کنشگرِ بسته‌بندِ سفارش:

- انتخابِ اندازهٔ جعبه ← `Order Fulfillment`
- علامت‌گذاریِ سفارش به‌عنوانِ آمادهٔ ارسال ← `Order Fulfillment`
- ارسالِ سفارش برای مشتری ← `Order Shipment`

کنشگرِ سیستم:

- تنظیمِ موجودیِ انبار ← `Inventory Management`
- سفارشِ موجودیِ بیشتر از تأمین‌کننده ← `Supplier Ordering`
- اعمالِ پرداخت ← `Order Payment`

همانند رویکرد گردش‌کار، هر کنش لزوماً مؤلفهٔ خودش را ندارد. برای نمونه، انتخابِ اندازهٔ جعبه برای سفارش و علامت‌گذاریِ آن به‌عنوانِ آمادهٔ ارسال، هر دو را مؤلفهٔ `Order Fulfillment` انجام می‌دهد.

به‌طورِ کلی، رویکرد کنشگر/کنش، بسته به اینکه معمار چند گردش‌کارِ اصلی را برای مدل‌سازی برگزیند، مؤلفه‌های بیشتری از رویکرد گردش‌کار تولید می‌کند. با این حال، در هر دو رویکرد، معمار می‌تواند مؤلفه‌های اصلیِ اولیه و شیوهٔ ارتباطِ آن‌ها را حتی پیش از دریافتِ الزامات یا مشخصاتِ تفصیلی شناسایی کند.

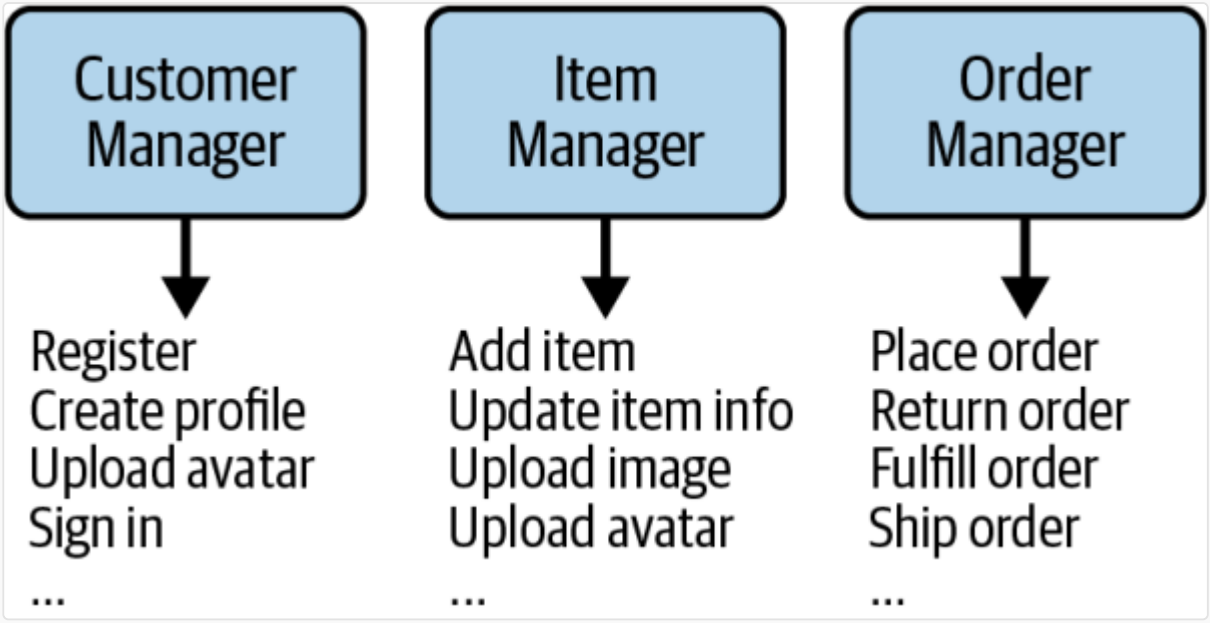
تلهٔ موجودیت

وسوسهٔ بسیار زیادی برای معمار وجود دارد که شناساییِ مؤلفه‌ها را با تمرکز بر موجودیت‌های درگیر در سیستم آغاز کند و سپس مؤلفه‌ها را از آن موجودیت‌ها استخراج کند. برای نمونه، در یک سیستمِ ثبتِ سفارشِ معمولی، ممکن است `Customer`، `Item` و `Order` را موجودیت‌های اصلیِ سیستم تشخیص دهید و به تناظرِ آن‌ها مؤلفه‌های `Customer Manager`، `Item Manager` و `Order Manager` را بسازید. به دلایلِ زیر قویاً از این رویکرد برحذر می‌داریم.

نخست، نام‌های این مؤلفه‌های منطقی مبهم‌اند و نقشِ مؤلفه را توصیف نمی‌کنند. برای نمونه، اگر فقط با نگاه به نامِ مؤلفهٔ `Order Manager` بپرسیم چه کاری انجام می‌دهد، پاسخِ بی‌فایدهٔ «سفارش‌ها را مدیریت می‌کند» به دست می‌آید که هیچ چیزی دربارهٔ نقش یا مسئولیتِ مشخصِ آن در سیستم نمی‌گوید. این را با نامِ مؤلفه‌ای مانندِ `Validate Order` مقایسه کنید تا اهمیتِ نامِ خوب و توصیفیِ مؤلفه روشن شود. اگر نامِ مؤلفه‌ای پسوندی مانندِ `Manager`، `Supervisor`، `Controller`، `Handler`، `Engine` یا `Processor` داشته باشد، نشانهٔ خوبی است که معمار ممکن است در دامِ ضدالگوی تلهٔ موجودیت افتاده باشد.

دوم، مؤلفه‌ها به انبارِ درهمی `(dumping ground)` برای کارکردهای مرتبط با حوزه تبدیل می‌شوند. نامِ مؤلفه‌ای موجودیت‌محور مانندِ `Order Manager` را در نظر بگیرید که در شکل ۸-۸ نشان داده شده است. هر ذره از کارکردهای مربوط به سفارش در همین یک مؤلفه جای می‌گیرد: اعتبارسنجیِ سفارش، ثبتِ سفارش، تاریخچهٔ سفارش، آماده‌سازیِ سفارش، ارسالِ سفارش، ردیابیِ سفارش و جز آن. این مؤلفه در اصل به آن کلاس‌های کمکیِ «همه‌چیزدار» `(kitchen sink)` شبیه می‌شود که هر توسعه‌دهنده‌ای دست‌کم یک بار در دورهٔ کاری‌اش نوشته است؛ کلاس‌هایی با ده‌ها متد که دست‌کاریِ رشته‌ها، دست‌کاریِ داده‌ها، محاسباتِ زمانی و هر چیزِ دیگری را که توسعه‌دهنده بتواند در آن بگنجاند انجام می‌دهند.

سوم، وقتی مؤلفه‌ها بیش از حد درشت‌دانه `(coarse-grained)` شوند، کارهای بیش از حد انجام می‌دهند و هدفشان را از دست می‌دهند. مؤلفه‌ها به‌جای آنکه ریزدانه `(fine-grained)` و تک‌منظوره باشند (مانندِ مثالِ مؤلفهٔ `Validate Order`)، ممکن است بیش از حد بزرگ شوند. نگهداری، آزمون و استقرارِ چنین مؤلفه‌هایی دشوار است و از این رو چندان قابل اعتماد نیستند.

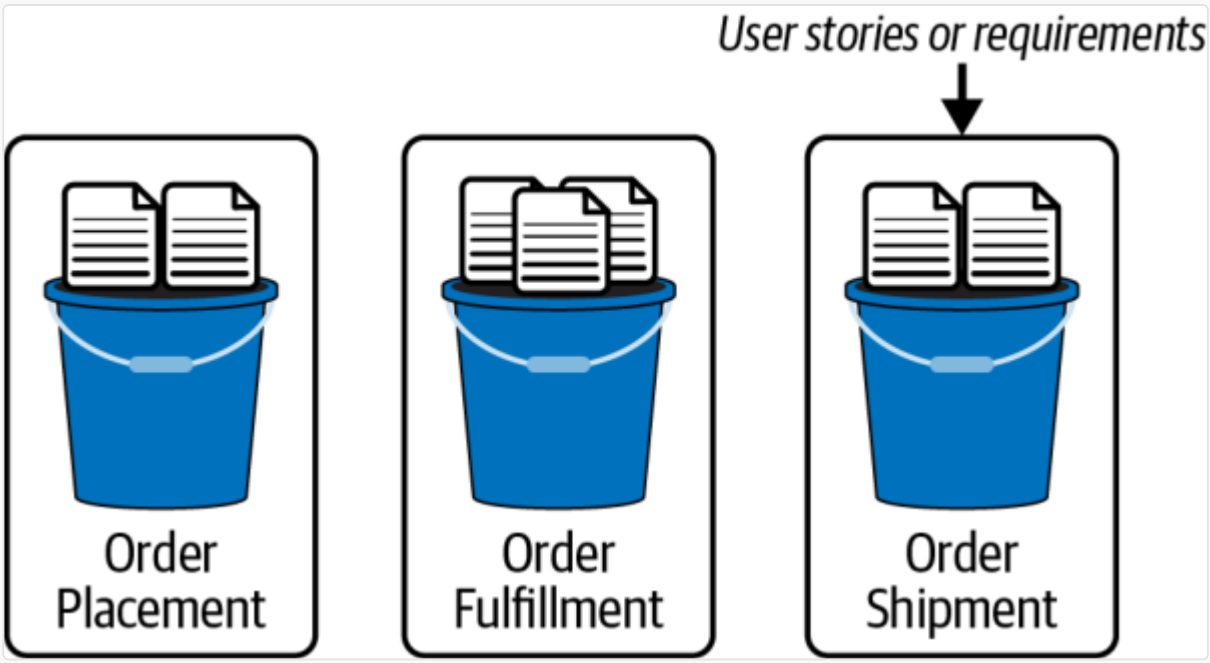


شکل ۸-۸. به‌کارگیریِ ضدالگوی تلهٔ موجودیت مؤلفه‌هایی با مسئولیتِ بیش از اندازه پدید می‌آورد

اگر معمار سیستمی می‌سازد که واقعاً موجودیت‌محور است و فقط عملیاتِ مبتنی بر `CRUD` (ایجاد، خواندن، به‌روزرسانی و حذف) را روی آن موجودیت‌ها انجام می‌دهد، آن سیستم به معماری نیاز ندارد؛ بلکه به یک چارچوب، ابزار یا محیطِ بدون‌کد/کم‌کد `(no-code/low-code)` مبتنی بر `CRUD` نیاز دارد که به توسعه‌دهندگان امکان دهد بیشترِ کدِ منبعی را که روی آن موجودیت‌ها عمل می‌کند تولید کنند.

تخصیصِ داستان‌های کاربری به مؤلفه‌ها

گامِ بعدی در ساختنِ معماری منطقی، تخصیصِ داستان‌های کاربری یا الزامات به مؤلفه‌های منطقی است. این فرایندی تکرارشونده است، زیرا بیشترِ داستان‌های کاربری یا الزامات از پیش به‌طورِ کامل شناخته‌شده نیستند؛ آن‌ها همراه با تکاملِ سیستم تکامل می‌یابند. هدفِ این گام آغازِ پرکردنِ آن سطل‌های خالی و دادنِ نقش‌ها و مسئولیت‌های مشخص به مؤلفه‌هاست؛ چنان‌که در شکل ۸-۹ نشان داده شده است.



شکل ۸-۹. پرکردنِ سطل‌های خالی (مؤلفه‌ها) با داستان‌های کاربری یا الزامات

برای اینکه ببینید مؤلفه‌های منطقی چگونه تکامل می‌یابند، داستان‌های کاربریِ زیر را در نظر بگیرید:

مشتریِ شمارهٔ ۱: به‌عنوانِ مشتری، می‌خواهم سفارشم اعتبارسنجی شود تا مطمئن شوم همه‌چیز را کامل و درست وارد کرده‌ام.

آماده‌کنندهٔ سفارش: به‌عنوانِ کسی که سفارش را آماده می‌کند، می‌خواهم بدانم برای سفارش از چه اندازه جعبه‌ای استفاده کنم تا بتوانم آن را به بهینه‌ترین شکلِ ممکن بسته‌بندی کنم.

مشتریِ شمارهٔ ۲: به‌عنوانِ مشتری، می‌خواهم هر بار که وضعیتِ سفارش تغییر می‌کند ایمیلی دریافت کنم تا همیشه از وضعیتِ سفارشم آگاه باشم.

فرض کنید معمار تاکنون مؤلفه‌های منطقیِ زیر را شناسایی کرده است:

- `Order Placement`
- `Order Fulfillment`
- `Order Shipment`
- `Inventory Management`

منطقی است که نخستین داستانِ کاربری را به مؤلفهٔ `Order Placement` تخصیص دهیم، زیرا کاربر برای ثبتِ سفارش با همین مؤلفه تعامل دارد:

> اعتبارسنجیِ سفارش (داستانِ کاربریِ مشتریِ شمارهٔ ۱) ← `Order Placement`

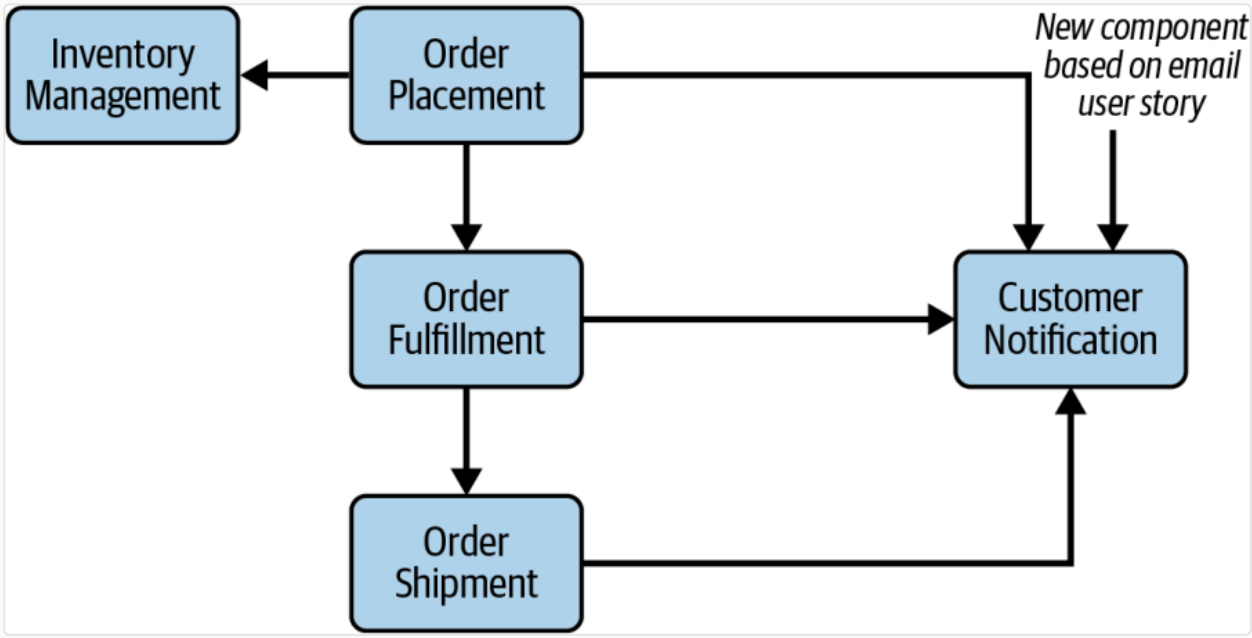
تعیینِ اندازهٔ جعبه را احتمالاً باید مؤلفهٔ `Order Fulfillment` بر عهده بگیرد، زیرا این مؤلفه مسئولِ همهٔ منطقِ سیستم برای آماده‌سازی و بسته‌بندیِ سفارش در جعبه است:

> تعیینِ اندازهٔ جعبه (داستانِ کاربریِ آماده‌کنندهٔ سفارش) ← `Order Fulfillment`

اما داستانِ کاربریِ سوم چه؟ کدام‌یک از چهار مؤلفهٔ فهرست‌شده باید هنگامِ ثبتِ سفارش، آماده‌شدنِ آن برای ارسال و ارسالِ آن، به مشتری ایمیل بفرستد؟ پاسخ ممکن است مؤلفه‌های `Order Placement`، `Order Fulfillment` و `Order Shipment` باشد. اما به یاد داشته باشید که داستانِ کاربری از راهِ کدِ منبع پیاده می‌شود و این کد باید در دایرکتوری یا فضای نامِ مشخصی قرار گیرد. از آنجا که تکثیرِ این کد در سه مؤلفه ایدهٔ خوبی نیست، معمار باید مؤلفهٔ جدیدی برای رسیدگی به این داستانِ کاربری تعریف کند:

> ایمیل به مشتری (داستانِ کاربریِ مشتریِ شمارهٔ ۲) ← `Customer Notification` (جدید)

مؤلفه‌های `Order Placement`، `Order Fulfillment` و `Order Shipment` باید با این مؤلفهٔ جدید ارتباط برقرار کنند تا به آن اطلاع دهند که ایمیلی بفرستد. با این افزوده، معماری منطقی اکنون مانندِ شکل ۸-۱۰ است.



شکل ۸-۱۰. تکاملِ مؤلفه‌ها بر پایهٔ داستان‌های کاربریِ جدید

تحلیلِ نقش‌ها و مسئولیت‌ها

گامِ بعدی در پالایشِ مؤلفه‌های منطقی، تحلیلِ نقش‌ها و مسئولیت‌های هر مؤلفه است. معمار از این راه اطمینان می‌یابد که الزامات یا داستان‌های کاربریِ تخصیص‌یافته به آن مؤلفه‌ها واقعاً به آن‌ها تعلق دارند و مؤلفه‌ها کارِ بیش از اندازه انجام نمی‌دهند. دغدغهٔ معمار در این گام انسجام است: اینکه عملیاتِ یک مؤلفه چگونه و تا چه اندازه با هم

مرتبط‌اند. با گذشتِ زمان، مؤلفه‌ها ممکن است بیش از حد بزرگ شوند، حتی اگر همهٔ عملیاتشان با هم مرتبط باشند.

برای نشان‌دادنِ کارکردِ این گام، فرض کنید معمار الزاماتِ زیر را به مؤلفهٔ `Order Placement` تخصیص داده است:

- اعتبارسنجیِ سفارش برای اطمینان از اینکه همهٔ فیلدها وارد شده‌اند و درست‌اند.
- نمایشِ سبدِ خرید `(shopping cart)` همراه با توضیحات، تعداد و قیمتِ اقلام.
- تعیینِ نشانیِ درستِ ارسال.
- گردآوریِ اطلاعاتِ پرداخت.
- تولیدِ یک شناسهٔ سفارشِ یکتا.
- اعمالِ پرداختِ سفارش.
- تنظیمِ موجودیِ انبار برای اقلامِ سفارش‌داده‌شده.
- ارسالِ خلاصهٔ سفارش برای مشتری با ایمیل.

اگر معمار بخواهد بیانیهٔ نقش و مسئولیتِ `(role and responsibility statement)` این مؤلفه را بنویسد، چنین خواهد بود:

> این مؤلفه مسئولِ اعتبارسنجیِ سفارش و نمایشِ سبدِ خریدِ معتبر، همراه با تصویر، توضیحات، تعداد و قیمتِ اقلام است. این مؤلفه همچنین مسئولِ تعیینِ نشانیِ درستِ ارسالِ سفارش و نیز گردآوریِ همهٔ اطلاعاتِ پرداخت از مشتری است. افزون بر این، اعمالِ پرداخت، تنظیمِ موجودیِ انبار و ارسالِ خلاصهٔ سفارش برای مشتری با ایمیل نیز بر عهدهٔ آن است.

هرچند همهٔ این عملیات به ثبتِ سفارش مربوط‌اند، مؤلفهٔ `Order Placement` آشکارا مسئولیتِ بیش از اندازه‌ای بر عهده گرفته است؛ به‌ویژه در زمینهٔ اعمالِ پرداخت، تنظیمِ موجودیِ انبار و ایمیل‌زدن به مشتری. یکی از راه‌های تشخیصِ اینکه آیا مؤلفه‌ای مسئولیتِ بیش از اندازه بر عهده گرفته است، جست‌وجوی عبارت‌های ربطی `(conjunctive phrases)` مانندِ «و»، «همچنین»، «افزون بر این» یا «و نیز» و استفادهٔ افراطی از ویرگول است.

از بخش‌های پیشینِ این فصل به یاد دارید که هر مؤلفهٔ منطقی با یک فضای نام یا دایرکتوری در مخزنِ کد نمایش داده می‌شود. در موردِ مؤلفهٔ `Order Placement`، همهٔ کدِ منبعِ این مؤلفه در یک دایرکتوری یا فضای نامِ واحد، مانندِ `com/app/order/placement` یا `com.app.order.placement`، قرار می‌گیرد. این حجمِ زیادی از کارکرد است؛ احتمالاً کدی بیش از اندازه برای یک دایرکتوریِ واحد. بنابراین منطقی است که فایل‌های کلاسِ مربوط به پردازشِ پرداخت، مدیریتِ موجودیِ انبار و ارتباطاتِ ایمیلی را در دایرکتوری‌های جداگانه‌ای قرار دهیم که نمایانگرِ همان کارکردها باشند. جداسازیِ مؤلفه‌های منطقی دقیقاً همین است.

اگر معمار مسئولیت‌های اعمالِ پرداخت، تنظیمِ موجودیِ انبار و ارسالِ ایمیل به مشتری را به مؤلفه‌های جداگانه منتقل کند، می‌تواند مسئولیتِ تک‌مؤلفهٔ `Order Placement` را کاهش دهد و نگهداری، آزمون و استقرارِ آن را آسان‌تر کند. مؤلفه‌های حاصل چنین خواهند بود:

`Order Placement`

- اعتبارسنجیِ سفارش برای اطمینان از اینکه همهٔ فیلدها وارد شده‌اند و درست‌اند.
- نمایشِ سبدِ خرید همراه با توضیحات، تعداد و قیمتِ اقلام.
- تعیینِ نشانیِ درستِ ارسال.
- گردآوریِ اطلاعاتِ پرداخت.
- تولیدِ یک شناسهٔ سفارشِ یکتا.

`Payment Processing`

- اعمالِ پرداخت.

`Inventory Management`

- تنظیمِ موجودیِ انبار برای اقلامِ سفارش‌داده‌شده.

`Customer Notification`

- ارسالِ خلاصهٔ سفارش برای مشتری با ایمیل.

اکنون هر مؤلفه نقش و مسئولیتِ روشن‌تر و متمایزتری دارد.

تحلیلِ ویژگی‌های معماری

گامِ پایانیِ تحلیل، در نظر گرفتنِ ویژگی‌های معماری‌ای است که سیستم به آن‌ها نیاز خواهد داشت. برخی ویژگی‌های معماری، مانندِ مقیاس‌پذیری، قابلیت اطمینان، در دسترس‌بودن، تحمل خطا، کشسانی و چابکی (توانِ پاسخ‌گوییِ سریع به تغییر)، ممکن است بر اندازهٔ یک مؤلفهٔ منطقی اثر بگذارند.

برای نمونه، شکستنِ یک مؤلفهٔ بزرگ‌تر (مؤلفه‌ای با مسئولیت‌های فراوان) به مؤلفه‌های کوچک‌تر، نگهداری و آزمونِ هر یک را آسان‌تر می‌کند (این همان چابکی است) و مقیاس‌پذیری، کشسانی و تحمل خطای بهتری فراهم می‌آورد. نمونهٔ خوبِ دیگر جایی است که دو بخش از سیستم با ورودیِ کاربر سروکار دارند: اگر یک بخش با صدها کاربرِ هم‌زمان سروکار داشته باشد و دیگری فقط باید از چند کاربر در هر لحظه پشتیبانی کند، این دو به ویژگی‌های معماریِ متفاوتی نیاز خواهند داشت. بنابراین، هرچند نگاهِ صرفاً کارکردی به طراحیِ مؤلفه ممکن است معمار را به تخصیصِ یک مؤلفهٔ واحد برای رسیدگی به تعامل با کاربر سوق دهد، تحلیلِ مؤلفه‌ها از منظرِ ویژگی‌های معماری ممکن است به تقسیمِ آن بینجامد.

از آنجا که معمار باید پیش از ساختنِ معماری منطقی ویژگی‌های معماری را بشناسد، این تحلیل معمولاً پس از تعیینِ مهم‌ترین ویژگی‌های معماریِ سیستم انجام می‌شود.

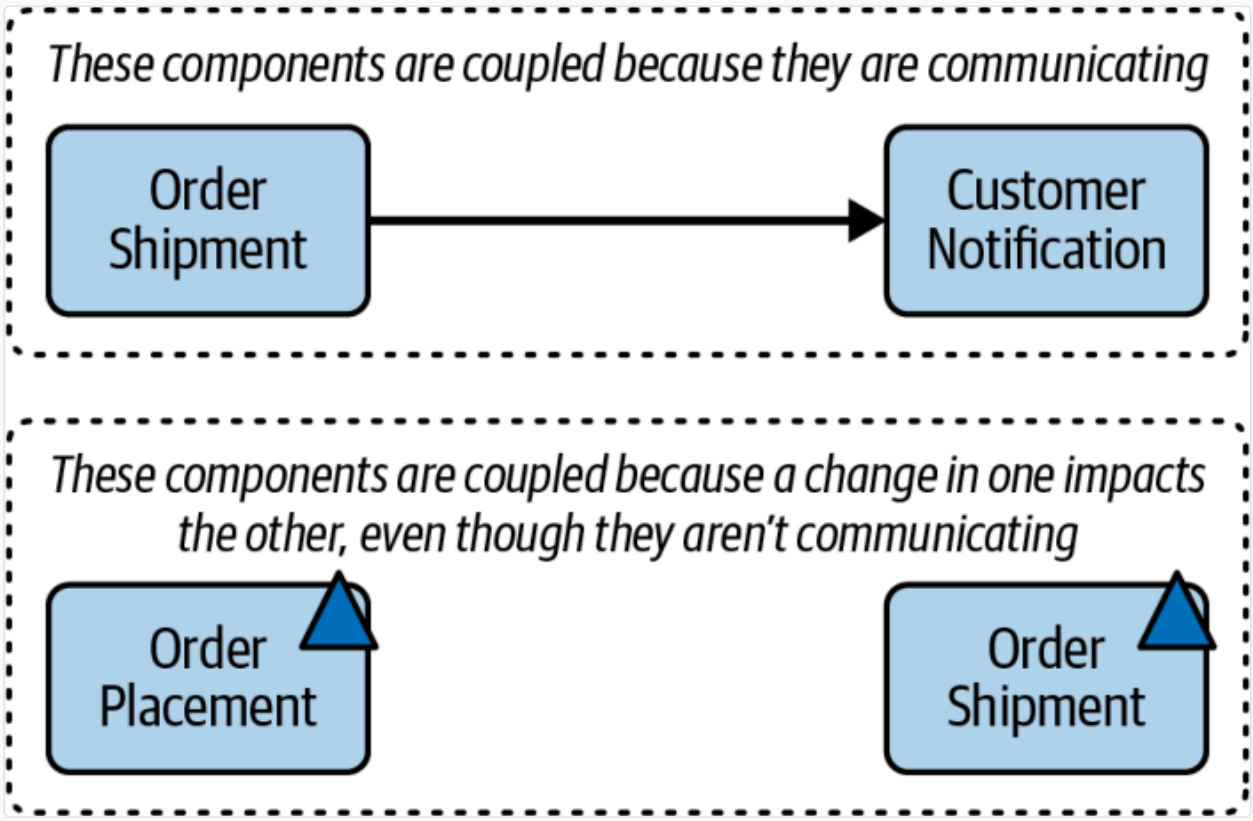
بازساختاردهیِ مؤلفه‌ها

بازخورد در طراحیِ نرم‌افزار حیاتی است. معماران باید همواره و در همکاری با توسعه‌دهندگان، طراحیِ مؤلفه‌هایشان را به‌صورتِ تکرارشونده بهبود دهند. طراحیِ نرم‌افزار انواعِ دشواری‌های پیش‌بینی‌نشده را پیش می‌آورد. از این رو، رویکردِ تکرارشونده در طراحیِ مؤلفه‌ها کلیدی است. نخست، تقریباً ناممکن است همهٔ کشف‌های گوناگون و حالت‌های مرزی `(edge cases)` را که پیش خواهند آمد و هر یک می‌تواند انگیزهٔ بازطراحی شود، از پیش در نظر گرفت. دوم، هرچه معماری و توسعه‌دهندگان در ساختنِ برنامه عمیق‌تر شوند، درکِ ظریف‌تری از اینکه رفتارها و نقش‌های برنامه باید کجا قرار گیرند به دست می‌آورند.

معماران باید انتظار داشته باشند که در سراسرِ چرخهٔ عمرِ `(lifecycle)` یک سیستم یا محصول، مؤلفه‌ها را بارها بازساختاردهی کنند؛ نه فقط در سیستم‌های نوساخت، بلکه در هر سیستمی که نگهداریِ مکرر دارد.

کوپلینگِ مؤلفه‌ها

وقتی مؤلفه‌ها با یکدیگر ارتباط برقرار می‌کنند، یا وقتی تغییر در یک مؤلفه ممکن است بر مؤلفه‌های دیگر اثر بگذارد، می‌گوییم آن مؤلفه‌ها با هم کوپل شده‌اند. هرچه مؤلفه‌های یک سیستم کوپلینگِ بیشتری داشته باشند، نگهداری و آزمونِ سیستم دشوارتر است (شکل ۸-۱۱ را ببینید). از این رو، توجهِ دقیق به کوپلینگ اهمیت دارد.

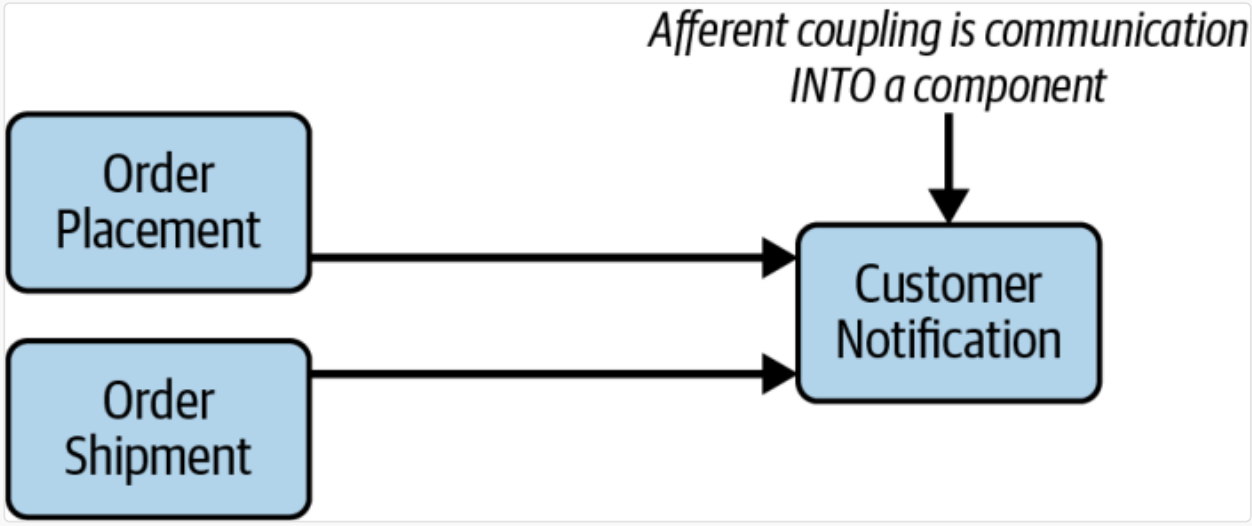


شکل ۸-۱۱. مؤلفه‌ها می‌توانند حتی بدونِ ارتباطِ مستقیم با هم کوپل باشند

کوپلینگِ ایستا

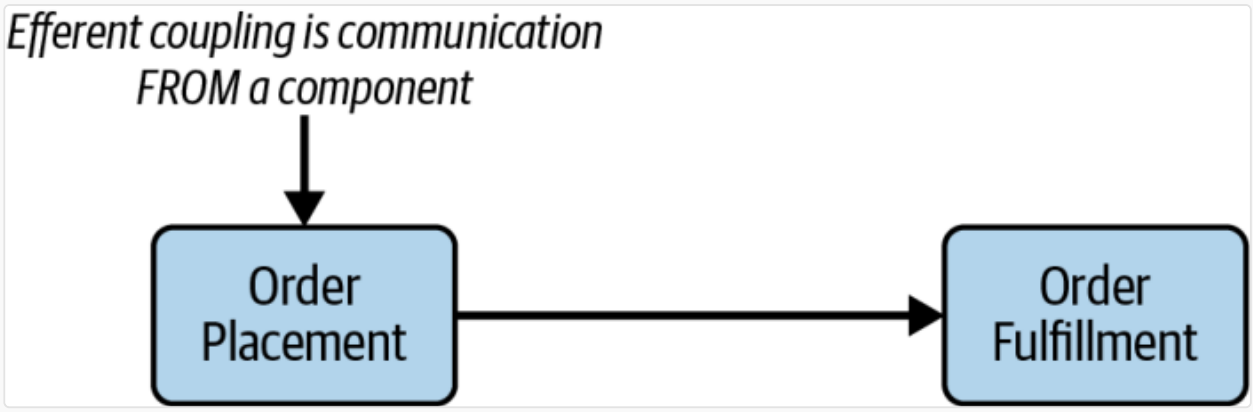
کوپلینگ ایستا زمانی رخ می‌دهد که مؤلفه‌ها به‌طورِ هم‌گام با یکدیگر ارتباط برقرار کنند. معماران باید به دو نوع کوپلینگ توجه کنند: ورودی و خروجی.

کوپلینگ ورودی (که کوپلینگِ درون‌سو `(incoming)` یا هم‌گرا `(fan-in)` نیز نامیده می‌شود) میزانِ وابستگیِ مؤلفه‌های دیگر به یک مؤلفهٔ هدف است. برای نمونه، مؤلفهٔ `Customer Notification` را در مثالِ ثبتِ سفارشِ ما در شکل ۸-۱۲ در نظر بگیرید. برای ایمیل‌زدن به مشتری، هر دو مؤلفهٔ `Order Placement` و `Shipment` باید با مؤلفهٔ `Customer Notification` ارتباط برقرار کنند. این یعنی مؤلفهٔ `Customer Notification` با مؤلفه‌های `Order Placement` و `Order Shipment` کوپلینگِ ورودی دارد و سطحِ کوپلینگِ ورودیِ آن ۲ است (شمارِ وابستگی‌های ورودی). کوپلینگ ورودی معمولاً با `CA` نشان داده می‌شود.



شکل ۸-۱۲. مؤلفهٔ `Customer Notification` با مؤلفه‌های دیگر کوپلینگِ ورودی دارد

کوپلینگ خروجی (که کوپلینگِ برون‌سو `(outgoing)` یا انشعابی `(fan-out)` نیز نامیده می‌شود) میزانِ وابستگیِ یک مؤلفهٔ هدف به مؤلفه‌های دیگر است. برای نمونه، چنان‌که در شکل ۸-۱۳ نشان داده شده، مؤلفهٔ `Order Placement` به مؤلفهٔ `Order Fulfillment` وابسته است؛ بنابراین با آن کوپلینگِ خروجی دارد و سطحِ کوپلینگِ خروجیِ آن ۱ است (شمارِ وابستگی‌های خروجی). کوپلینگ خروجی معمولاً با `CE` نشان داده می‌شود.



شکل ۸-۱۳. مؤلفهٔ `Order Placement` با مؤلفهٔ `Order Fulfillment` کوپلینگِ خروجی دارد

کوپلینگِ زمانی

کوپلینگ زمانی `(temporal coupling)` وابستگی‌های غیرایستا را توصیف می‌کند؛ معمولاً وابستگی‌هایی که بر پایهٔ زمان‌بندی یا تراکنش‌ها `(transactions)`، یعنی واحدهای کارِ منفرد، شکل می‌گیرند. برای نمونه، وقتی سیستم سفارشی را پردازش می‌کند، کارکردِ مؤلفهٔ `Order Placement` باید پیش از کارکردِ مؤلفهٔ `Order Shipment` فراخوانی شود. از این رو می‌گوییم این مؤلفه‌ها کوپلینگِ زمانی دارند.

مشکلِ کوپلینگ زمانی این است که تشخیصِ آن با ابزارهای موجود در بازار دشوار است. در بیشترِ موارد، این نوع کوپلینگ به‌جای آن از راهِ اسنادِ طراحی شناسایی می‌شود یا از راهِ وضعیت‌های خطا آشکار می‌گردد.

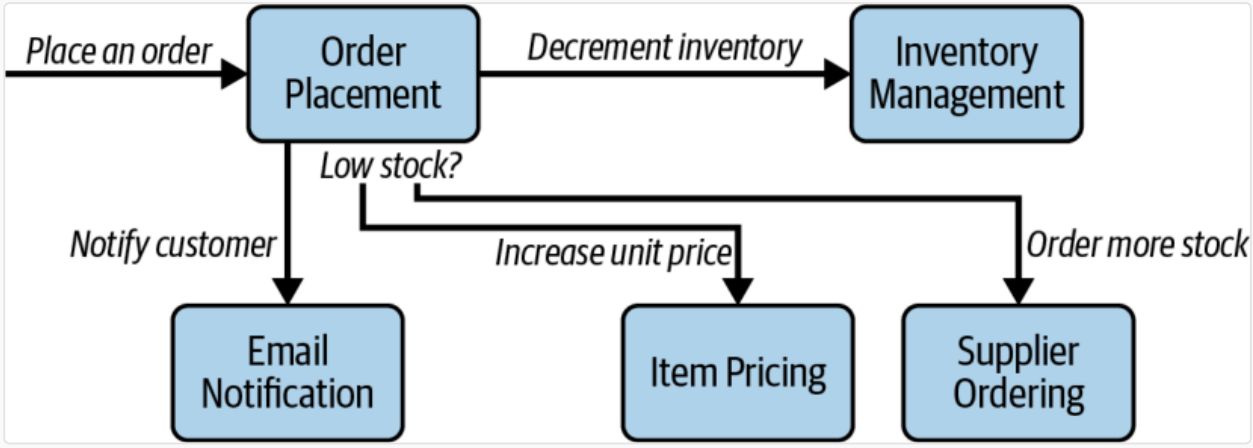
قانونِ `Demeter`

به بیشترِ معماران توصیه می‌شود که در طراحیِ سیستم برای کوپلینگ سست بکوشند. کوپلینگِ کمتر میانِ مؤلفه‌ها یا سرویس‌ها، نگهداریِ سیستم را آسان‌تر، آزمونش را ساده‌تر و استقرارش را کم‌خطرتر می‌کند. همچنین قابلیت اطمینانِ کلیِ سیستم را افزایش می‌دهد، زیرا تغییرات بر مؤلفه‌های کمتری اثر می‌گذارند و فرصت‌های کمتری برای خطا پدید می‌آید.

یکی از تکنیک‌های ساختنِ سیستم‌های با کوپلینگِ سست، قانونِ `Demeter` `(Law of Demeter)` نام دارد که اصلِ کمترین دانش `(principle of least knowledge)` نیز خوانده می‌شود. در اسطوره‌های یونانی، الههٔ `Demeter` همهٔ غلهٔ جهان را تولید می‌کرد، اما هیچ نمی‌دانست مردم با آن چه می‌کنند (به دام‌ها می‌خورانند، نان می‌پزند و جز آن). `Demeter` در عمل از بقیهٔ جهان جدا `(decoupled)` بود.

قانونِ `Demeter` می‌گوید: یک مؤلفه یا سرویس باید دانشِ محدودی از مؤلفه‌ها یا سرویس‌های دیگر داشته باشد. هرچند این گفته ممکن است ساده‌انگارانه و بدیهی به نظر برسد، چنین نیست. بگذارید منظورمان را نشان دهیم.

مؤلفه‌ها و ارتباط‌های متناظرشان را که در شکل ۸-۱۴ نشان داده شده‌اند در نظر بگیرید. پس از پذیرشِ سفارشِ مشتری، مؤلفهٔ `Order Placement` باید به مؤلفهٔ `Inventory Management` بگوید که موجودیِ انبار را کاهش دهد. اگر موجودی بیش از حد پایین بیاید، مؤلفهٔ `Order Placement` باید دو کار انجام دهد: به مؤلفهٔ `Supplier Ordering` بگوید موجودیِ بیشتری از تأمین‌کننده سفارش دهد و به مؤلفهٔ `Item Pricing` بگوید قیمت را بر پایهٔ عرضهٔ محدودِ موجود تنظیم کند. سرانجام، وقتی همهٔ این کارها انجام شد، مؤلفهٔ `Order Placement` به مؤلفهٔ `Email Notification` می‌گوید جزئیاتِ سفارش را برای مشتری ایمیل کند.



شکل ۸-۱۴. مؤلفهٔ `Order Placement` با بقیهٔ سیستم کوپلینگِ بالایی دارد، زیرا دانشِ بیش از اندازه‌ای دارد

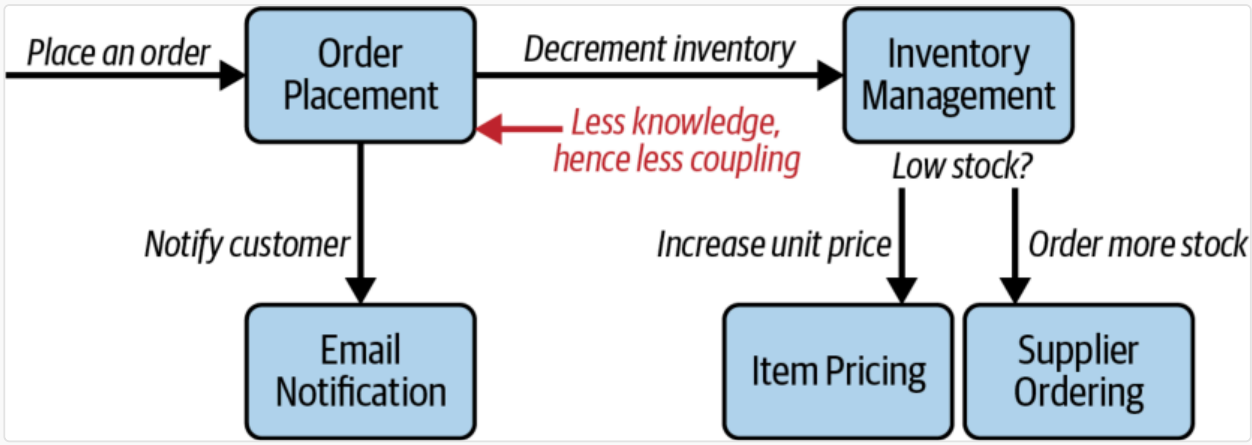
دقت کنید که در این معماری، مؤلفهٔ `Order Placement` با مؤلفه‌های دیگر کوپلینگِ بالایی دارد. هرچند مسئولیتِ انجامِ آن کنش‌ها را ندارد، از این دانش برخوردار است که این کنش‌ها باید رخ دهند؛ و دانشِ بیشتر یعنی کوپلینگِ تنگاتنگ‌تر.

ایدهٔ پشتِ قانونِ `Demeter` محدودکردنِ دانشِ هر مؤلفه دربارهٔ بقیهٔ سیستم است. در شکل ۸-۱۴، مؤلفهٔ `Order Placement` می‌داند که موجودی باید کاهش یابد، شاید لازم باشد موجودیِ بیشتری سفارش داده شود، شاید قیمتِ کالا باید تنظیم شود و ایمیلی باید برای مشتری فرستاده شود. (این دانشِ زیادی است!) اما اگر بتوان این دانش را میانِ مؤلفه‌های دیگر توزیع کرد چه؟ در آن صورت، معمار می‌تواند آن مؤلفه را عملاً از بقیهٔ سیستم جدا کند.

برای اینکه ببینید قانونِ `Demeter` چگونه می‌تواند برای کاهشِ کوپلینگِ مؤلفه‌ها به کار رود، به سیستمِ شکل ۸-۱۴ برگردید. اگر معمار مؤلفهٔ دیگری میانِ `Order Placement` و `Inventory Management` می‌افزود تا دانشِ لزومِ کاهشِ موجودی را به آن واگذارد، مؤلفهٔ `Order Placement` همچنان همان سطحِ کوپلینگ خروجی (برون‌سو) را داشت. بنابراین آن نقطهٔ کوپلینگ باید همان‌گونه که هست باقی بماند.

اما دانشِ اینکه اگر عرضه بیش از حد پایین بیاید، سیستم باید موجودیِ بیشتری سفارش دهد و قیمتِ کالا را تنظیم کند چه می‌شود؟ هر دوِ این دانسته‌ها را می‌توان به مؤلفهٔ `Inventory Management` واگذار کرد و از این راه سطحِ کوپلینگِ مؤلفهٔ `Order Placement` را کاهش داد. شکل ۸-۱۵ معماریِ حاصل از به‌کارگیریِ قانونِ `Demeter` را

نشان می‌دهد. حذفِ دانشِ لزومِ رخدادنِ برخی کارکردها، کوپلینگِ مؤلفهٔ `Order Placement` با بقیهٔ سیستم را کاهش می‌دهد.



شکل ۸-۱۵. مؤلفهٔ `Order Placement` وقتی دانشِ کمتری داشته باشد، کوپلینگِ کمتری با سیستم دارد

خوانندهٔ تیزبین درمی‌یابد که هرچند به‌کارگیریِ قانونِ `Demeter` سطحِ کوپلینگِ مؤلفهٔ `Order Placement` را کاهش داد، سطحِ کوپلینگِ مؤلفهٔ `Inventory Management` را افزایش داد. به‌کارگیریِ قانونِ `Demeter` لزوماً سطحِ کوپلینگ را در کلِ سیستم کاهش نمی‌دهد؛ بلکه معمولاً آن کوپلینگ را میانِ بخش‌های مختلفِ سیستم بازتوزیع می‌کند.

مطالعهٔ موردی: `Going, Going, Gone` — کشفِ مؤلفه‌ها

اگر تیمی محدودیتِ خاصی نداشته باشد و به دنبالِ یک تجزیهٔ مؤلفه‌ای `(component decomposition)` خوب و همه‌منظوره باشد، رویکرد کنشگر/کنش به‌عنوانِ راه‌حلی عمومی به‌خوبی جواب می‌دهد.

اگر معمار رویکرد کنشگر/کنش را بر `Going, Going, Gone` `(GGG)` اعمال کند، درمی‌یابد که این سیستم سه نقشِ اصلی دارد: پیشنهاددهنده `(bidder)`، حراج‌گردان `(auctioneer)` و سیستم؛ که این آخری در این تکنیکِ مدل‌سازی مشارکت‌کننده‌ای همیشگی برای کنش‌های درونی است. این نقش‌ها با کنش‌های اصلیِ زیر با سیستم تعامل دارند:

`Bidder`

- مشاهدهٔ جریانِ زندهٔ ویدئو.
- مشاهدهٔ جریانِ زندهٔ پیشنهادها.
- ثبتِ پیشنهاد.

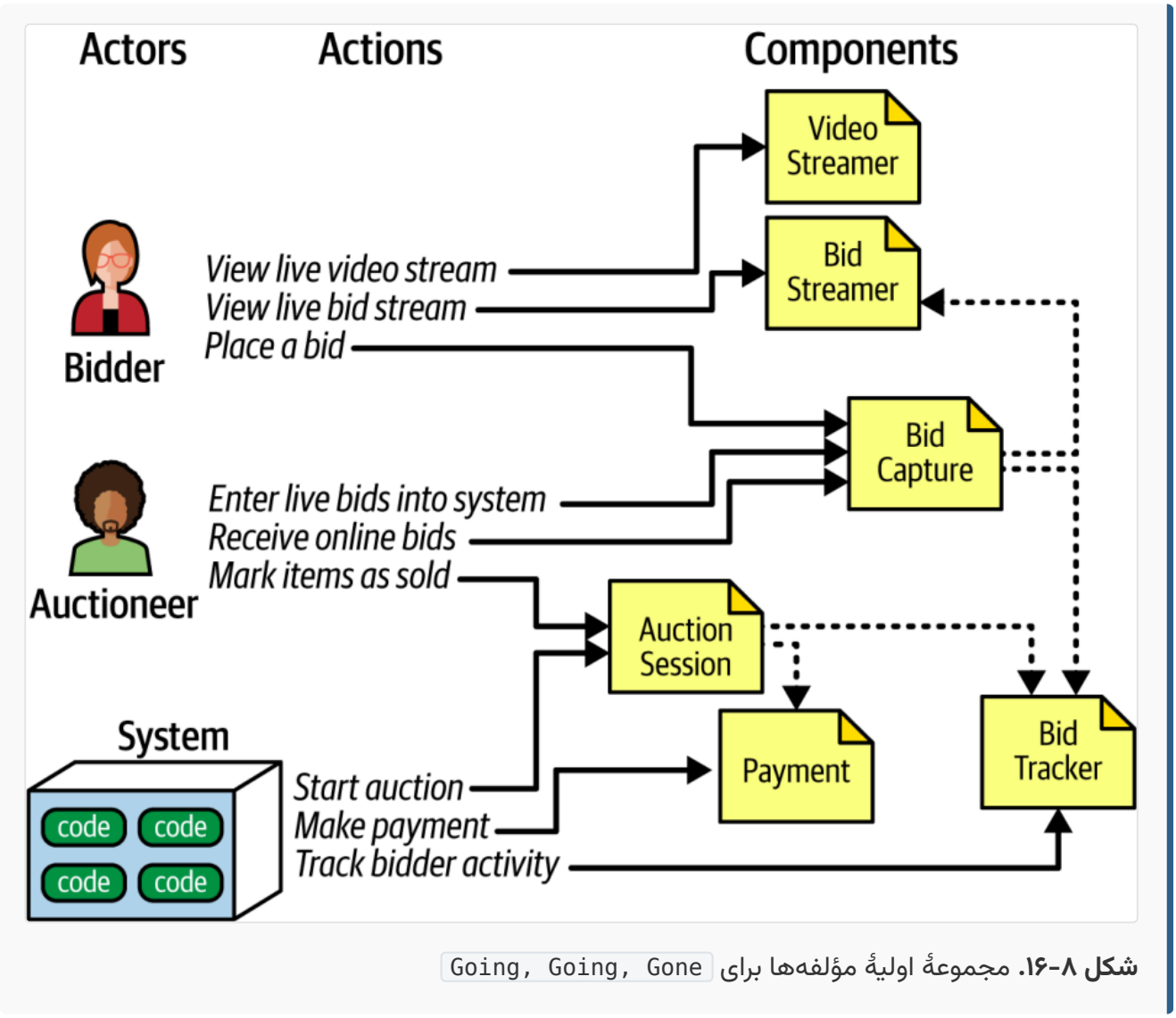
`Auctioneer`

- واردکردنِ پیشنهادهای حضوری به سیستم.
- دریافتِ پیشنهادهای برخط.
- علامت‌گذاریِ قلم کالا به‌عنوانِ فروخته‌شده.

`System`

- آغازِ حراج.
- انجامِ پرداخت.
- ردیابیِ فعالیتِ پیشنهاددهندگان.

با در دست داشتنِ این کنش‌ها، معمار می‌تواند مجموعه‌ای از مؤلفه‌های آغازین برای `GGG` بسازد و سپس آن را به‌صورتِ تکرارشونده بهبود دهد. یکی از این راه‌حل‌ها در شکل ۸-۱۶ آمده است.



شکل ۸-۱۶. مجموعهٔ اولیهٔ مؤلفه‌ها برای `Going, Going, Gone`

در شکل ۸-۱۶، هر نقش و کنش به یک مؤلفه نگاشت می‌شود. مؤلفه‌ها ممکن است برای اشتراکِ اطلاعات به همکاری نیاز داشته باشند. مؤلفه‌هایی که برای این راه‌حل شناسایی کردیم عبارت‌اند از:

`Video Streamer`: حراجِ زنده را به‌صورتِ جریانی برای کاربران پخش می‌کند.

`Bid Streamer`: پیشنهادها را هم‌زمان با ثبتشان به‌صورتِ جریانی برای کاربران پخش می‌کند. `Video Streamer` و `Bid Streamer` هر دو نمایی فقط‌خواندنی `(read-only)` از حراج در اختیارِ پیشنهاددهندگان می‌گذارند.

`Bid Capture`: این مؤلفه پیشنهادها را از حراج‌گردان و پیشنهاددهندگان دریافت و ثبت می‌کند.

`Bid Tracker`: پیشنهادها را ردیابی می‌کند و نقشِ سیستمِ مرجع `(system of record)` را دارد.

`Auction Session`: حراج را آغاز می‌کند. وقتی پیشنهاددهنده برنده می‌شود و حراجِ آن قلم کالا پایان می‌یابد، این مؤلفه گام‌های پرداخت و تسویه را به راه می‌اندازد؛ از جمله اطلاع‌دادن به پیشنهاددهندگان دربارهٔ قلمِ بعدیِ حراج.

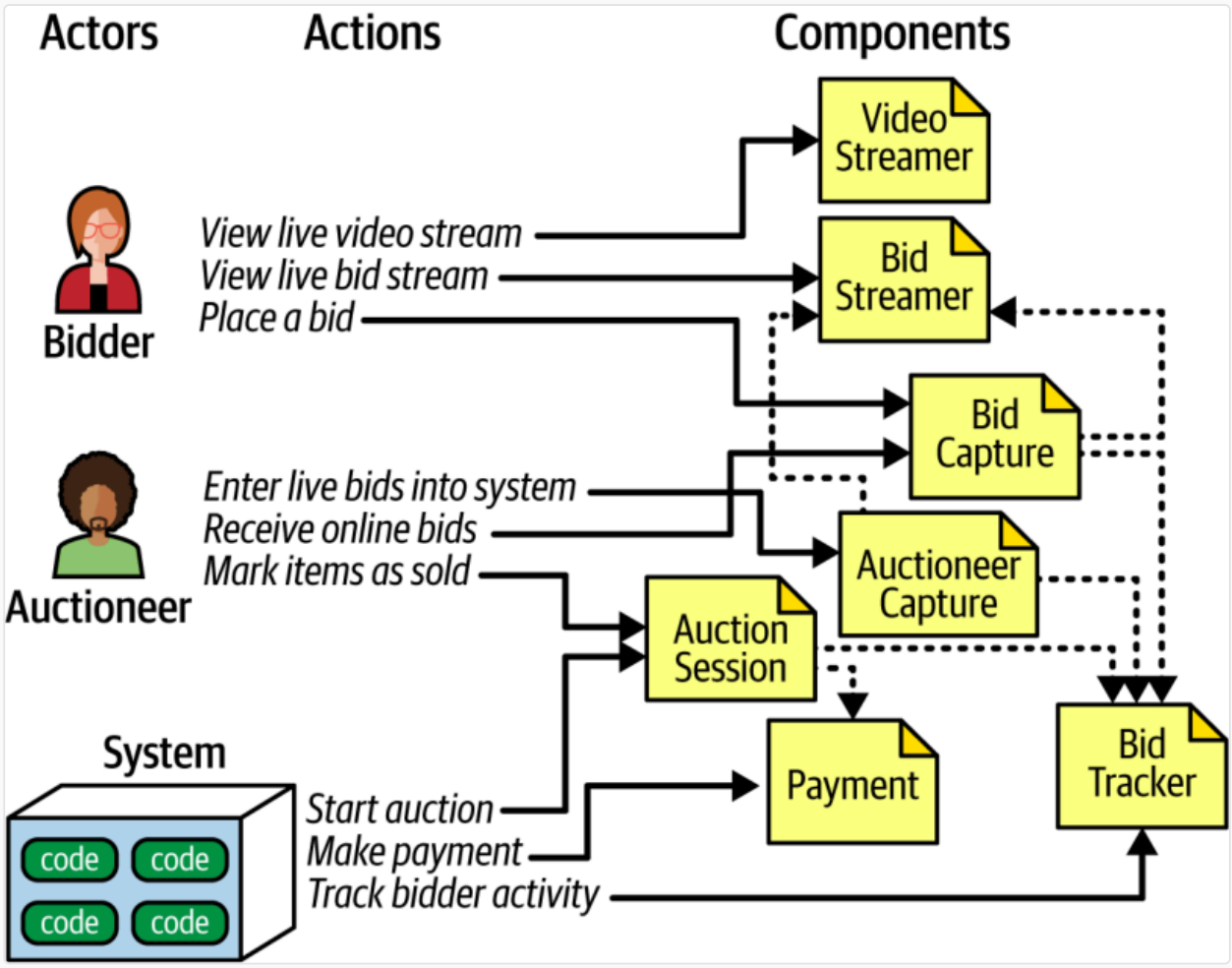
`Payment`: پردازشگرِ پرداختِ شخص ثالث برای پرداخت با کارت اعتباری.

پس از دورِ نخستِ شناسایی مؤلفه‌ها (شکل ۸-۶ را ببینید)، معمار ویژگی‌های معماری‌ای را که پیش‌تر شناسایی شده بودند تحلیل می‌کند تا دریابد آیا هیچ‌یک از آن‌ها طراحیِ مؤلفه را تغییر می‌دهد یا نه. برای نمونه، طراحیِ کنونی مؤلفهٔ `Bid Capture` را برای دریافتِ پیشنهادها هم از پیشنهاددهندگان و هم از حراج‌گردان در نظر گرفته است. این از نظرِ کارکردی منطقی است: پیشنهادِ هر کسی را می‌توان به یک شیوه دریافت و پردازش کرد. اما دریافتِ پیشنهادها به کدام‌یک از ویژگی‌های معماریِ شناسایی‌شده نیاز دارد؟ حراج‌گردان به همان سطحِ مقیاس‌پذیری یا کشسانیِ پیشنهاددهندگان نیاز ندارد که شمارشان ممکن است به هزاران نفر برسد.

به همین ترتیب، حراج‌گردان ممکن است به برخی ویژگی‌های معماریِ شناسایی‌شده بیش از دیگر بخش‌های سیستم نیاز داشته باشد؛ مانندِ قابلیت اطمینان (اطمینان از اینکه اتصال‌ها قطع نمی‌شوند) و در دسترس‌بودن (اطمینان از اینکه سیستم بالا و فعال می‌ماند). برای نمونه، هرچند برای کسب‌وکار بد است که پیشنهاددهنده‌ای نتواند وارد سایت شود یا اتصالش قطع شود، اگر یکی از این دو برای حراج‌گردان رخ دهد فاجعه‌بار خواهد بود.

معمار برای پشتیبانی از سطوحِ متفاوتِ نیازِ پیشنهاددهندگان و حراج‌گردان به ویژگی‌های معماریِ یکسان، تصمیم می‌گیرد مؤلفهٔ `Bid Capture` را به دو مؤلفه تقسیم کند: `Bid Capture` و `Auctioneer Capture`. طراحیِ به‌روزشده در شکل ۸-۱۷ آمده است.

معمار مؤلفهٔ جدیدی برای `Auctioneer Capture` می‌سازد. او همچنین پیوندهای اطلاعاتی را از `Auctioneer Capture` به `Bid Streamer` (برای نمایشِ پیشنهادهای حضوری به پیشنهاددهندگانِ برخط) و به `Bid Tracker` (برای مدیریتِ جریان‌های پیشنهاد) به‌روز می‌کند. اکنون `Bid Tracker` مؤلفه‌ای است که دو جریانِ اطلاعاتیِ بسیار متفاوت را یکپارچه می‌کند: جریانِ واحدِ اطلاعاتِ حراج‌گردان و جریان‌های متعددِ پیشنهاددهندگان.



شکل ۸-۱۷. مؤلفه‌های بازنگری‌شدهٔ مطالعهٔ موردیِ `GGG`

بعید است طراحیِ نشان‌داده‌شده در شکل ۸-۱۷ طراحیِ نهایی باشد. الزاماتِ بیشتری باید کشف شوند: اینکه افراد چگونه حسابِ کاربریِ جدید ثبت خواهند کرد، کارکردهای پرداخت چگونه اداره خواهند شد و جز آن. با این حال، این طراحی نقطهٔ آغازِ خوبی برای تکرارهای بعدی فراهم می‌کند.

این یکی از مجموعه‌های ممکنِ مؤلفه‌ها برای حلِ مسئلهٔ `GGG` است؛ اما لزوماً نه بهترینِ آن‌هاست و نه تنها راه‌حل. سیستم‌های نرم‌افزاریِ اندکی هستند که فقط به یک شیوه پیاده‌سازی‌شدنی باشند. هر طراحی مجموعه‌های متفاوتی از مصالحه‌ها دارد. به‌عنوانِ معمار، درگیرِ وسواسِ یافتنِ «یگانه طراحیِ درست» نشوید، زیرا طراحی‌های بسیاری کفایت می‌کنند. بکوشید مصالحه‌های میانِ تصمیم‌های طراحیِ مختلف را تا جای ممکن عینی ارزیابی کنید و طراحی‌ای را برگزینید که «کم‌بدترین» `(least worst)` مجموعهٔ مصالحه‌ها را دارد.

واژه‌نامهٔ اختصاصی این فصل

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Component-Based Thinking	تفکر مؤلفه‌محور
Logical Component	مؤلفهٔ منطقی
Building Block	بلوک سازنده
Floor Plan	نقشهٔ طبقه
Leaf Node	گرهٔ برگ
Subdomain	زیرحوزه
Logical Architecture	معماری منطقی
Physical Architecture	معماری فیزیکی
Actor	کنشگر
Repository	مخزن
Physical Artifact	مصنوع فیزیکی
Deployment Unit	واحد استقرار
Component Identification	شناسایی مؤلفه‌ها
Candidate Component	مؤلفهٔ نامزد
Feedback Loop	حلقهٔ بازخورد
Core Component	مؤلفهٔ اصلی
Refactoring	بازآرایی
Greenfield System	سیستم نوساخت
Placeholder	جانگهدار
Workflow Approach	رویکرد گردش‌کار
Actor/Action Approach	رویکرد کنشگر/کنش
Entity Trap	تلهٔ موجودیت

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Happy Path	مسیر موفق
User Journey	سفر کاربری
Order Packer	بسته‌بند سفارش
Dumping Ground	انبار درهم
Kitchen Sink (Utility Class)	کلاس کمکی همه‌چیزدار
Coarse-Grained	درشت‌دانه
Fine-Grained	ریزدانه
CRUD (Create, Read, Update, Delete)	CRUD (ایجاد، خواندن، به‌روزرسانی، حذف)
No-Code/Low-Code	بدون‌کد/کم‌کد
User Story	داستان کاربری
Role and Responsibility Statement	بیانیهٔ نقش و مسئولیت
Conjunctive Phrase	عبارت ربطی
Shopping Cart	سبد خرید
Fault Tolerance	تحمل خطا
Concurrent Users	کاربران هم‌زمان
Restructuring (Components)	بازساختاردهی (مؤلفه‌ها)
Edge Case	حالت مرزی
Lifecycle	چرخهٔ عمر
Component Coupling	کوپلینگ مؤلفه‌ها
Afferent Coupling	کوپلینگ ورودی
Incoming (Fan-in) Coupling	کوپلینگ درون‌سو (هم‌گرا)
Efferent Coupling	کوپلینگ خروجی
Outgoing (Fan-out) Coupling	کوپلینگ برون‌سو (انشعابی)

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Temporal Coupling	کوپلینگ زمانی
Transaction	تراکنش
Law of Demeter	قانون Demeter
Principle of Least Knowledge	اصل کمترین دانش
Decoupled	جدا (فاقد کوپلینگ)
Component Decomposition	تجزیهٔ مؤلفه‌ای
Bidder	پیشنهاددهنده
Auctioneer	حراج‌گردان
Read-Only	فقط‌خواندنی
System of Record	سیستم مرجع
Third-Party	شخص ثالث
Least Worst (Trade-offs)	کم‌بدترین (مجموعهٔ مصالحه‌ها)